

Clos / Fat-Tree Data-Centre Fabrics

A feasibility study: can it frame the SYN-flood thesis?

Working note — my2 project

17 September 2026

Purpose. This note answers, from trusted sources, every question raised about the Clos (fat-tree) data-centre topology: what it is, whether it is real, where servers and hosts sit, why and when it is used, whether it has already been used against SYN floods, and—most importantly—whether it gives an *honest* home for this project’s distributed detection + controller reconstruction. The verdict is at the end; the hard parts are not hidden.

1 Is the Clos / fat-tree topology real? **YES**

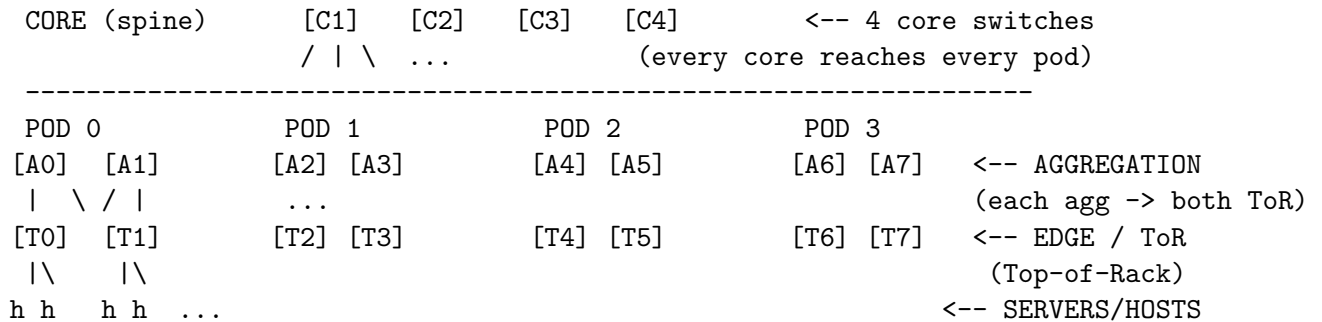
It is the **dominant network topology inside real data centres**. It is not a toy.

- The idea comes from **Charles Clos (1953)**, originally for telephone switching fabrics.
- It was adapted to data centres by **Al-Fares, Loukissas, Vahdat, “A Scalable, Commodity Data Center Network Architecture”, SIGCOMM 2008** — the paper that introduced the *fat-tree* (a folded Clos built from cheap commodity switches).
- It is what **Google, Meta/Facebook, Microsoft, Amazon** actually run inside their data centres today. A “spine-leaf” network (the term vendors use) is a 2-tier Clos.

So building on it is the opposite of a toy: it is the standard against which real in-network systems are measured.

2 What it looks like (structure)

A fat-tree is a **multi-rooted tree** with three switch layers and **many equal-length parallel paths** between any two servers. A “ k -ary” fat-tree has k pods; here is $k = 4$ (20 switches, 16 hosts):



- **ToR / edge (leaf):** each rack of servers plugs into one Top-of-Rack switch.
- **Aggregation:** connects the ToRs within a pod to the core.
- **Core / spine:** the roots; every pod reaches every core switch.
- **Key property — full bisection bandwidth:** between any two servers there are $(k/2)^2$ equal-cost paths (for $k = 4$, four paths). No matter where two servers are, they can talk at line speed. That is the whole reason the topology exists.

3 Where are the servers and hosts? And the “replica” question

Servers hang off the ToR switches at the bottom. The Clos is the *fabric* (the switches); the servers are the leaves. So “data centres use Clos” and “there are servers” are both true at once: the switches form the Clos, the servers plug into the ToRs.

Do multiple servers hold replicas of the same data? **YES**

A public service (say a website) is **not one machine**. It is:

- advertised to the world as **one Virtual IP (VIP)** — one address clients resolve via DNS;
- backed by **many replica servers**, each with its own real address (**DIP**, direct IP), spread across different ToRs/racks for load and fault tolerance.

“Why would a host TCP-connect with one *or more* servers?” — the correction

Your intuition is close but one detail matters: **a single TCP connection always goes to exactly one server**. A connection is point-to-point; a client never pulls one connection’s data from several servers at once. What actually happens is:

1. The client opens a connection to the **VIP** (it thinks it is talking to one machine).
2. An edge router / load balancer **hashes the connection’s 5-tuple** and picks *one* backend replica (DIP) to handle it. All packets of that connection go to that one replica (kept together so TCP does not reorder).
3. A *different* connection (different 5-tuple) is hashed to a *different* replica.

So it is: **one connection** → **one server**; **many connections** → **spread across many replica servers** by the load balancer. Replies often go *straight back* to the client (Direct Server Return), bypassing the balancer so it does not become a bottleneck. This is exactly how a SYN flood on a VIP gets **fanned out across many backend servers and many fabric paths** — which is the interesting part for you.

4 Why and when is Clos used?

- **Full bisection bandwidth** at **moderate cost** using commodity switches (Al-Fares’ headline result) — any-to-any line-rate communication.
- **Massive multipath** — many equal-cost paths give load balancing and **fault tolerance** (a switch/link dies, traffic reroutes).
- **Scale-out** — add pods to grow; no single expensive “big” core box.

When: inside data centres / clusters where east–west (server-to-server) traffic is huge. *Not* a wide-area map like GÉANT — see the contrast box.

Contrast — your real GÉANT files. The three files in `my2/geant/` are the genuine Topology Zoo graphs: **Geant2012 = 40 nodes, 61 edges**, and the nodes are *countries* (Austria, Belgium, ..., United Kingdom), i.e. one PoP per national research network. GÉANT is a **wide-area backbone**; Clos is a **data-centre fabric**. They are different worlds, and my earlier 8-node “LON/AMS/FRA” sketch was invented, *not* real GÉANT.

5 How traffic takes paths in Clos — the crux for the thesis

This one section decides whether the topology helps you. In a Clos there are many parallel paths, so the fabric must choose one per packet. There are three schemes, and they behave very differently:

Scheme	What it does	Does one flow fragment?
Per-flow ECMP (the default)	Hash the 5-tuple → one path. All packets of a connection (SYN <i>and</i> ACK) take the <i>same</i> path. Chosen to avoid TCP reordering.	NO — a whole flow stays on one path; one switch sees it entirely.
Per-packet spraying	Each packet independently picks a random path. Best load balance, but causes re-ordering (TCP sees duplicate ACKs, cuts its window). Used only in special fabrics.	YES — a single flow’s packets scatter across switches.
Flowlet switching	Spray <i>bursts</i> (flowlets) separated by idle gaps; finer than per-flow, less reordering than per-packet. The practical middle ground (e.g. CONGA).	Partly — bursts of a flow land on different paths.

Read this carefully, because it is the whole answer:

- Under the **default (per-flow ECMP)**, a connection’s SYN and its ACK have the same 5-tuple → same path → **one switch sees both**. Your controller’s per-flow ACK reconstruction is then **dead weight** — exactly the problem you already spotted. *But* a single *source’s* flood uses **many source ports** (each SYN is a different 5-tuple), so those SYNs **do** scatter across many switches. No single switch sees the source’s *total* → the controller aggregating per-source counts across switches is **genuinely necessary and real**.
- Under **per-packet spraying**, even a single connection’s SYN and ACK land on *different* switches → no switch sees the whole flow → your **EVIDENCE reconstruction becomes necessary and honest** (no flag-based splitter needed). The price: spraying is a specific design choice and causes reordering.

6 Has Clos already been used against SYN floods? **YES** — and you must know this

There is strong, recent prior work doing in-network DDoS/SYN defence on programmable (P4) switches in data-centre / ISP fabrics. Be honest about it in the thesis:

- **Poseidon** (NDSS 2020) — maps volumetric-DDoS defences (including SYN flood) to programmable switches; used in a “scrubbing centre”.
- **Jaquen** (USENIX Security 2021) — switch-native detection *and* mitigation entirely in the data plane for volumetric attacks in ISPs; reports line-rate, and **0% false positives vs Poseidon’s 25.2% in SYN-proxy**, using 6× less memory.
- **P4 SYN-proxy** works (TU Munich, 2020) — SYN-cookie / proxy in the data plane.

Implication: the “P4 switch defends a data centre against SYN flood” space is *occupied* by heavy-weight work. You are *not* first, and Jaquen already claims 0% FP. So a Clos framing only works if your contribution is clearly *different* from these (see the verdict).

7 Can you frame the thesis on Clos? Where would detection sit?

Where the detectors go: the natural place is the **aggregation or spine (core) layer**, because those switches carry *aggregated* east–west traffic from many ToRs — a flood spread across the fabric passes through them, and (crucially) **each spine sees only a fraction** of it (ECMP spreads flows

across spines). That fragmentation is what makes distributed detection + controller aggregation *necessary*, not decorative.

Where the hosts/servers go: victim servers under one or more ToRs (behind a VIP). Attackers are either external (entering through the core/border) or internal compromised hosts under *other* ToRs.

The honest test (the only one that matters):

*Does the topology fragment one attacker's SYN evidence across multiple switches, **without reading the TCP flag**, so that no single switch can decide alone?*

- **Per-source count aggregation: YES**, for free, under normal ECMP (a source's many SYNs scatter by source port). This justifies your *controller aggregation* honestly.
- **Per-flow SYN/ACK reconstruction (EVIDENCE):** only **YES** if you assume **per-packet spraying**. Under default ECMP it is **NO**.

8 Advantages and disadvantages for the thesis

Advantages

- Real, standard, defensible topology — kills the “your topology is a toy” criticism outright.
- Under ECMP, a source's flood *genuinely* fragments across switches → distributed detection + controller aggregation is **honestly motivated** (no artificial flag-splitter).
- With a per-packet-spraying assumption, even your EVIDENCE reconstruction becomes legitimate.
- Natural fit for your validated *windowed + reputation-score* mitigation (the part that is genuinely yours and already gives 0% FP / 1-pps floor).

Disadvantages / risks (do not hide these)

- **Strong prior work** (Poseidon, Jaqen). Jaqen already reports 0% FP with a SYN-proxy in the data plane. Your novelty must be crisp (the reputation-hysteresis mitigation and the distributed cross-switch reconstruction), and your comparison honest.
- **The EVIDENCE reconstruction needs per-packet spraying** to be necessary — a specific assumption you must state and defend (spraying is real but not universal; it causes reordering).
- **Scale on your box:** a $k=4$ fat-tree is 20 BMv2 switches + 16 hosts on WSL — heavy but runnable; $k=6$ (45 switches) will crawl. Keep it small.
- **Addressing:** a fat-tree uses structured (pod-based) addressing; mapping your single-subnet IPv6 L2 design onto it needs care (still doable at L2, but not free).

9 Proposed testbed design (concrete)

A $k=4$ fat-tree, small enough to run on BMv2/WSL, large enough to be a real Clos. **20 switches**, plus hosts and servers on the leaves.

Switch inventory — who is smart, who is dumb

Layer	Count	Role	Pipeline
Spine (core)	4	dumb forwarder	P4, routes between pods only. No CMS, no digests.
Aggregation	8	SMART detector	P4 <i>with</i> CMS + FIRST.SEEN/THRESHOLD/EVIDENCE digests. This is where detection lives.
ToR / edge	8	dumb forwarder + per-packet spray	P4, sprays each packet across its two aggregation uplinks (round-robin). No detection.

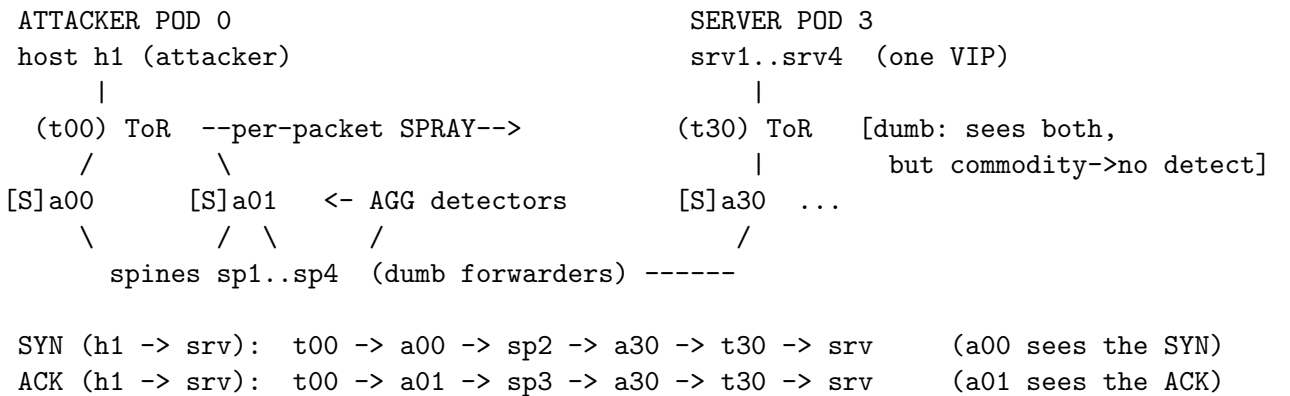
Why the detectors sit at aggregation (your instinct, and it's right): the ToR sprays a connection's packets across its two aggregation uplinks, so the SYN and the ACK of the *same* connection land on *different* aggregation switches. The aggregation layer is therefore exactly where one switch sees a fragment of a flow and *cannot* decide alone — forcing the controller. The ToR (which the traffic funnels through) is kept **dumb/commodity** so it never trivially sees the whole flow; that models the real world (commodity edge, programmable core) and removes the “why not just detect at the edge?” objection.

Hosts and servers (leaves)

Pod	Endpoints	Role
Pod 0	~6 hosts	Attackers (SYN flood via scapy, low + high rate)
Pod 1	~3 atk + 3 benign	Mixed attackers + legitimate clients
Pod 2	~6 hosts	Benign clients (ApacheBench)
Pod 3	4 servers	Victim data-centre rack behind one VIP (4 replica servers)

Totals: **20 switches**, ~24 endpoints (4 servers, ~10 attackers, ~10 benign).

The mechanism, proven — SYN and ACK on different paths to the same server



Result: in the attacker's own pod, **a00 sees only the SYN**, **a01 sees only the ACK** — two different smart switches, neither with the full picture. Only the controller, aggregating their digests, reconstructs “SYN without a matching ACK = half-open” and blocks the source *at the aggregation, near the attacker, before the flood reaches the server*. That is the mechanism, and here it is **necessary** because the spray genuinely split the flow — *no flag-based splitter*. (The server's ToR t30 does see both, but it is a dumb commodity leaf and never detects.)

Why the SYN and ACK diverge yet still reach the SAME server

This is the property that makes it work, and it is fundamental to fat-trees: routing happens in **two phases that behave oppositely**.

- **Going UP (host $\rightarrow$ core): a free choice.** There are many equal-cost ways up — any aggregation, any spine — because every spine can reach every pod. **This is the only place we spray.** It does not matter which spine a packet climbs to.
- **Going DOWN (core $\rightarrow$ server): fixed by the destination.** From *any* spine there is exactly *one* way down to a given server’s pod $\rightarrow$ aggregation $\rightarrow$ ToR $\rightarrow$ server. The down-path is dictated by the destination address, not chosen.

So the destination is never at risk: **scatter the upward path freely; the downward routing always delivers to the correct server**, whichever spine you came through.

```
SYN: h1 -> t00 --(spray up)--> a00 -> sp2 --(down: fixed by dst)--> a30 -> t30 -> srv
ACK: h1 -> t00 --(spray up)--> a01 -> sp3 --(down: fixed by dst)--> a31 -> t30 -> srv
      ^^^ up-halves DIFFER (spray) ^^^          ^^^ both end at SAME srv ^^^
```

The up-halves differ (SYN via a00/sp2, ACK via a01/sp3); both down-halves end at `srv` because its address forces the route. They converge only at the server’s own ToR `t30` (dumb) and the server. So each aggregation detector sees only a fragment, and only the controller ties them together.

Honest caveat: the split is *statistical*, not per-connection guaranteed. Round-robin alternates packets, and other traffic passes between a connection’s SYN and its ACK, so **roughly half** of connections end up with SYN and ACK on different aggregations (the rest happen to coincide). That is fine and realistic — it produces exactly the “some detectors have partial views” case the reconstruction handles. Forcing *every* connection to split would require routing on the flag again (the artificial thing we are escaping), so statistical divergence via spray is the honest version, and it is what real sprayed fabrics actually produce.

Watch-out to test first: per-packet spraying reorders *legitimate* handshakes too, so verify benign ab still completes (else benign FP rises). If it hurts, spray **flowlets** (bursts) instead of single packets, or let the reputation score absorb it.

10 Papers to compare against (name them explicitly)

A. Distributed / data-plane track (the Clos experiment)

- **Jaqen** — Liu et al., *USENIX Security 2021*. Switch-native volumetric DDoS detection + SYN-proxy mitigation. Compare *qualitatively* (regime: volumetric vs low-rate; technique: proxy vs detect-and-attribute; their reported 0% FP is a proxy guarantee, not detection).
- **Poseidon** — Zhang et al., *NDSS 2020*. Maps volumetric-DDoS defences (incl. SYN) to programmable switches; scrubbing centre. Qualitative.
- **SHIELD-P4** — Carvalho et al., *CANDARW 2025*. P4 data-plane trust + controller aggregation; edge switch distributes by source-IP range (a splitter). Closest to you; qualitative + possibly partial quantitative.
- **(optional) Edge-centric IPv6 zero-trust** — Aljoby et al., *arXiv 2026*. P4/BMv2, IPv6, CMS windowed counting; explicitly lists “SYN/ACK imbalance” as future work.

B. Symmetric / entropy track (the GÉANT + per-flow-ECMP measurement)

These are controller-centralised and **re-implementable**, so you get a *quantitative* head-to-head on your own testbed (FPR/FNR at 10–80% attack rates, single- and multi-victim):

- **SynFloWatch** — Sinha, *ICDCN 2024*. Tsallis entropy ($q=2$) on destination IP at the controller (Floodlight); Geant Zoo, 40 switches, hping3 + iperf. **Your base paper** — the primary quantitative baseline.
- **SAFETY** — Kumar, Tripathi, Nehra, Conti, Lal, “SAFETY: Early Detection and Mitigation of TCP SYN Flood Utilizing Entropy in SDN,” *IEEE TNSM*, vol. 15(4), 2018. Shannon-entropy baseline that SynFloWatch itself compares against. Second quantitative baseline.

Comparison plan. Symmetric track (GÉANT, per-flow ECMP): quantitative vs SynFloWatch and SAFETY (both entropy, both runnable). Distributed track (mini-Clos, per-packet spray): your distributed detector + reputation mitigation, positioned *qualitatively* against Jaqen/Poseidon/SHIELD-P4 on regime and technique — because those are un-runnable Tofino/ISP systems and no reviewer expects you to re-run them.

11 Verdict

- **Can you use Clos? Yes** — and it is a *much* stronger, more honest home than the artificial flag-splitter, because a source’s flood fragments across the fabric for real.
- **What survives cleanly:** distributed per-source detection with controller aggregation (justified by ECMP), plus your windowed + reputation-score mitigation (your genuine contribution).
- **What is conditional:** the per-flow EVIDENCE reconstruction — only necessary under a stated **per-packet-spraying** assumption. Decide whether you want to make that assumption or drop the reconstruction and lean on per-source aggregation.
- **What you must handle:** position yourself clearly against Poseidon and Jaqen. Your edge is the *low-false-positive reputation mitigation* and the *cross-switch reconstruction*, not “first to detect SYN floods on P4”.

Bottom line. Clos does not magically save the flag-splitter idea, but it *does* give your distributed architecture a real, published, defensible reason to exist — and it lets you keep the parts of your work that are genuinely strong. It is a better foundation than GÉANT for *this* detector, provided you (a) state the load-balancing assumption honestly and (b) differentiate from Jaqen.